

A-Tune-Online: Efficient and QoS-aware Online Configuration Tuning for Dynamic Workloads

Yu Shen^{†*}, Beicheng Xu^{†*}, Yupeng Lu[†], Donghui Chen[◇], Huaijun Jiang^{†‡}, Zhipeng Xie[◇],
Senbo Fu[◇], Nan Zhang[◇], Yuxin Ren[◇], Ning Jia[◇], Xinwei Hu[◇], Bin Cui[†]

[†]School of CS & Key Laboratory of High Confidence Software
Technologies (MOE), Peking University, Beijing, China

[◇]Huawei Technologies, Hangzhou, China

[‡]Center for Data Science, Peking University, Beijing, China

[†]{shenyu, xinkelyp, jianghuaijun, bin.cui}@pku.edu.cn, [†]beichengxu@stu.pku.edu.cn

Abstract—Automatic configuration tuning of online services with dynamic workloads has attracted increasing interest. Effective online tuning ensures configurations adapt to workload changes over time to maintain optimal online service performance. To be practical, online tuning must satisfy the dynamicity, efficiency, and Quality of Service (QoS) requirements. However, existing online tuning approaches fail to meet these requirements due to the inability to eliminate negative effects from historical observations.

In this paper, we propose A-Tune-Online, an online configuration tuning system that tackles dynamic workloads, delivering superior tuning efficiency, and QoS guarantee simultaneously to a wide range of online scenarios. We identify that restarting the optimization based on explicit workload shift detection is necessary and critical to eliminate negative historical observations. First, to invoke optimization restarts appropriately, we design a multi-stage multi-indicator detection strategy based on heuristic rules and configuration replays. Then, to avoid initial efficiency drop after re-optimization, A-Tune-Online utilizes a similarity-based dual warm start scheme that transfers knowledge from similar historical workloads effectively. Finally, to prevent transient performance degradation from violating QoS guarantee after optimization restart, we leverage lower confidence bound to construct a safety region where each configuration is expected to perform better than the QoS requirement. Empirical study on five tuning scenarios showcases the superiority of A-Tune-Online compared with state-of-art tuning systems. A-Tune-Online achieves an average speedup of 2.90x and 1.72x compared with OnlineTune and DDPG+, respectively. We provide a version of our system in <https://github.com/PKU-DAIR/A-Tune-Online>.

Index Terms—Automatic online configuration tuning, Universal scenarios, Bayesian Optimization, Workload shift detection.

I. INTRODUCTION

Automatic configuration tuning [1]–[8] spares the software engineers with the requirement of expert knowledge on a variety of configurable parameters (i.e., knobs) and improves the performance of complex software, including databases, compilers, system kernels, etc. Recently, the demand for tuning configurations of online services that handle dynamic workloads has significantly increased in the industry [8]–[11]. Static offline tuning is ineffective to improve online services, because the previous best configuration found statically may not be suitable for the current workload due to workload shifts.

* Yu Shen and Beicheng Xu contribute equally to this paper.

For example, the optimal knobs are quite different for read-only and write-only database workloads [9]. The dynamicity of the environment makes online configuration tuning more important but difficult.

In practice, the online tuning should fulfill the following goals to support dynamic workloads robustly and effectively. 1) **Dynamicity**. The online tuning system should be able to find near-optimal configurations that adapt to dynamic workloads over time to maximize service performance. Thus, workload changes should be correctly identified with high precision and recall rate. 2) **Efficiency**. Efficient tuning is a key requirement in online scenarios because a slow tuning process not only increases the tuning cost but also delays the service to enjoy the improved configurations. Therefore, online tuning should quickly adapt to new workloads, i.e., the algorithm should not be greatly affected by workload changes. 3) **QoS Guarantee**. Quality of Service (QoS) is the primary concern for customers in the industry. It requires the system to always deliver a specified level of online service performance to users, and online services are unacceptable for performance below the QoS guarantee, even for brief periods. Thus, online tuning must avoid transient performance degradation during the tuning process.

Unfortunately, existing configuration tuning solutions fail to effectively and practically satisfy online tuning requirements. On the one hand, directly applying offline tuning to online services is not feasible. Offline tuning usually assumes that workloads never shift and takes hours or days to find the best configurations, which may not be suitable for the current workload due to workload shifts [9]. On the other hand, it is non-trivial to step from offline to online tuning. The root cause is that existing online tuning methods fail to eliminate historical observations of dissimilar workloads, which negatively affect tuning performance and efficiency. Some pioneering online tuning works [2], [8] only support tuning specific service workloads undergoing slight change over time, but can't guarantee the performance when the workloads change significantly. Although OnlineTune [9] uses unsupervised clustering to cluster historical observations and trains a surrogate model for each cluster, misclassified observations still leave it misled by historical observations, degrading tuning efficiency

and performance (up to 13.2% in our experiments in §V).

We present A-Tune-Online, a novel online configuration tuning system that supports a range of application scenarios with dynamic workloads. The key idea in A-Tune-Online is to restart optimization to eliminate negative historical observations from previous dissimilar workloads. To avoid unnecessary restarts or the omission of necessary ones, we utilize explicit workload shift detection to select the optimal timing for optimization restarts. However, merely selecting the appropriate timing for re-optimization is insufficient to meet all online tuning requirements. During early iterations after re-optimization, limited observations will lead to unreliable surrogate model fitting and bad configuration sampling while exploring the highly uncertain space. Therefore, it is challenging to ensure efficiency and QoS guarantee of re-optimization.

A-Tune-Online integrates three key techniques to solve the above design challenges. 1) A-Tune-Online employs a multi-stage workload shift detection strategy. Specifically, we combine a rule-based multi-indicator detector and a replay-based detector to achieve high precision and recall. 2) A-Tune-Online utilizes a similarity-based dual warm start scheme to leverage the knowledge from previous similar tasks, enhancing early tuning iterations after re-optimization. 3) A-Tune-Online leverages lower confidence bound to build a safety region that avoids QoS violation during optimization.

We summarize our contributions as follows,

- We showcase the characteristics of online workload dynamics, identify fundamental efficiency issues in existing online tuning approaches and motivate the need to restart the optimization to tackle dynamic workloads.
- To deal with workload dynamics, we propose A-Tune-Online, an online tuning system that adopts workload shift detection and re-optimization to avoid being misled by previous dissimilar workloads.
- To achieve high efficiency and guarantee QoS after re-optimization, we reuse the knowledge of similar previous workloads and maintain a safety region to constrain the search space above the QoS limitations.
- We evaluate A-Tune-Online under various realistic workloads and scenarios, demonstrating the effectiveness and efficiency of our design. Concretely, it achieves an average speedup of 2.90x and 1.72x compared with OnlineTune and DDPG+, respectively.

II. BACKGROUND AND MOTIVATION

A. Workload Characteristic and Motivating Scenarios

Based on our industry products and operational experience, the real-world online workload has a notable characteristic: *it exhibits periodic or phased changes*, that is, one workload will run stably for a period of time and then transition to another workload. Therefore, even though the online system has diverse workloads, each workload will run steadily for some time. As discussed in §IV, this feature guides us in designing and optimizing A-Tune-Online. We list some specific practical scenarios that can benefit from A-Tune-Online.

1) *Regular software updates* introduce changes in algorithms, features, versions, or resource quota that can significantly impact performance characteristics of the system, necessitating tuning configuration. 2) *Phased transition between daytime and nighttime workloads*. It is common to shift workload between daytime and nighttime, with daytime operations dominated by interactive ad-hoc queries that demand low latency while nighttime operations focused on batch processing that requires high throughput. For example, our large language model GPU clusters serve inference requests in the daytime and are used for training at nighttime. This shift necessitates different configuration strategies. 3) *Trending topics of special events* often cause sudden increases in user traffic on social media platforms like Weibo [12]. These spikes require rapid scaling of resources and configuration adjustment to ensure service availability and performance.

B. Limitations of Existing Online Tuning Methods

Recently, some techniques have been proposed to tune the online performance of system applications by using the real workload directly instead of tuning on a static copy of the workload. For example, Pioneering work [2], [8] extends Bayesian optimization so that it can tune Spark SQL workloads that change slightly over time. OnlineTune [9] characterizes database workloads using information involving queries, tables, and operators, which limits the universality. Then, it trains performance models to fit the relationship between workloads and tuned performance. However, the performance of existing online tuning works is not guaranteed when the workloads change significantly. We identify two common issues in existing online tuning methods.

Issue #1: Existing approaches suffer from degraded performance and efficiency due to the negative impact of historical observations. Dissimilar workloads usually exhibit extremely poor correlation in performance. We conduct a quantitative study in Fig. 11(b), which shows negative Kendall-tau correlation among dissimilar workloads. Thus, using historical observations of dissimilar workloads introduces unavoidable negative impacts during online tuning.

Unfortunately, existing methods fail to completely eliminate poor historical observations. One category of online tuning methods, such as Rover [2] and Online-Tune [8], assumes minor workload changes and continuously uses all observation data for optimization. As a result, the current tuning will be misled by historical observations of shifted workloads, requiring a large number of new observations to gain an accurate understanding of the current workload. Another work, OnlineTune [9], uses DBSCAN to cluster all historical observations and trains a surrogate model for each cluster. However, the instability of unsupervised clustering leads to inaccurate segmentation, treating some observations as noise or forcing them into wrong clusters, which hinders tuning efficiency and performance. For example, during the 2400-minute MySQL tuning process, the mean mutual information between the classification results of observation performed by OnlineTune and the actual workload labels of the observations

TABLE I: Comparison between A-Tune-Online and existing works.

Criterion	A-Tune-Online	OnlineTune	Rover
Workload Features Generality	✓	✗	✓
Explicit Workloads Detection	✓	○	✗
Tuning Efficiency	✓	○	✗
Tuned Performance	✓	○	✗
QoS Guarantee	✓	✓	✗

at each tuning is 0.59, while A-Tune-Online achieves 0.88 with shift detection and similar task matching. Then, OnlineTune transfers through fitting surrogate models with historical observations deemed similar, which risks persistent impact from misclassified observations if similarity matching fails, resulting in sub-optimal performance. As evaluated in §V, misclassified observations degrade best-tuned performance by up to 13.20% in OnlineTune compared with A-Tune-Online.

Conclusion #1: Avoiding negative historical observations is necessary in online tuning; thus we need to restart the optimization to isolate configurations of dissimilar tasks.

Issue #2: Existing approaches cannot effectively restart the optimization due to the lack of explicit and accurate workload shift detection. A naive method of applying re-optimization to existing online tuning systems is restarting at a predetermined interval, but this lacks robustness and transferability. On the one hand, the optimizer can trigger unnecessary re-optimization if restarting during the same workload, and the algorithm has to re-converge. According to experiments in §V, with history transfer, MySQL requires an average of 15.6 configuration recommendations to reach the optimum. Therefore, an erroneous restart substantially increases the final convergence tuning cost (e.g., 156 minutes if tuning every 10 minutes). On the other hand, failing to execute a desired restart upon a significant workload change will mislead the current optimization, as explained in Issue #1, resulting in reduced tuning efficiency and the final best-tuned performance. As a result, the restart of optimization needs to be consistent with workload shifts to avoid unnecessary restarts during stable workloads and ensure continued optimization during shifts.

Conclusion #2: Explicit workload shift detection with high precision and recall is crucial to effectively utilize re-optimization in online tuning.

Table I compares A-Tune-Online with some leading online tuning methods. OnlineTune requires detailed query and data features, which are hard to generalize and obtain, while Rover uses more accessible and general runtime metrics. Rover assumes stable workloads, resulting in poor performance during shifts. OnlineTune handles shifts but still struggles with the negative effects of historical observations, leading to sub-optimal efficiency and performance. Rover does not consider QoS. OnlineTune uses black-box and white-box knowledge to establish safety regions, but negative historical observations still may cause transient QoS violation. In §III and §IV, we will introduce how A-Tune-Online overcame these problems of existing systems.

C. Bayesian Optimization Background

Bayesian optimization (BO) [13]–[15] is a popular algorithm framework that solves black-box problems with a costly objective function. Typically, BO loops over the following three main steps: 1) it fits a *surrogate model* M based on observed configurations $D = \{(x_1, y_1), \dots, (x_{n-1}, y_{n-1})\}$, where x_i is the i^{th} configuration and y_i is the corresponding observed performance; 2) with the surrogate model M , BO chooses the next configuration to evaluate by maximizing the *acquisition function* $x_n = \arg \max_{x \in \mathcal{X}} \alpha(x; M)$. 3) BO evaluates the configuration x_n to obtain its performance y_n and augment the observations by $D = D \cup \{(x_n, y_n)\}$.

Surrogate. We use the Probabilistic Random Forest [14] as the surrogate, since it achieves better empirical results in real-world scenarios due to low time complexity [16].

Acquisition function. We apply the Expected Improvement (EI) [17] function to estimate the performance improvement of an unseen configuration x . Given the marginal predictive mean $\mu(x)$ and variance $\sigma^2(x)$ by the surrogate M , the EI function is defined as the expected improvement over the best performance found, i.e., $EI(x) = \int_{-\infty}^{\infty} \max(y - y^*, 0) p_M(y|x) dy$, where y^* is the best performance observed so far.

Initialization. Before training the surrogate model with sufficient observations, we default to initialize BO by sampling several configurations using Latin Hypercube Sampling from the search space to generate near-random configurations.

III. A-TUNE-ONLINE OVERVIEW

A. Problem Definition

The objective of performance tuning is to identify a near-optimal configuration x^* within a specified search space. A configuration x is a vector $[x_{p_1}, x_{p_2}, \dots, x_{p_N}]$, where each p_i represents a tunable parameter, such as the buffer pool size in MySQL. Before optimization, users provide a search space $\mathcal{X}$ for these parameters, with each parameter having a value range like $[0, 1]$. Online tuning should deal with **dynamicity** and **QoS**. Dynamicity means that workload changes over time and leads to performance discrepancies when evaluating the same configuration at different time. QoS requires that the service performance during the tuning process cannot be worse than a QoS guarantee δ , which is typically the obtained performance without online tuning [18]. We define the configuration space where performance is above δ as the safety region. Formally, an online tuning problem is defined as follows,

$$x_t^* = \arg \max_{x \in \mathcal{X}} \mathcal{L}_t(x), \text{ s.t. } \mathcal{L}_t(x) \geq \delta \quad (1)$$

where $\mathcal{L}_t$ is the objective at the t^{th} iteration.

B. Key Insight and Design Challenges

We propose A-Tune-Online as a novel online configuration tuning system that supports a range of practical scenarios where the workload changes periodically over time. As analyzed in §II-B, to achieve high tuning efficiency, performance, and QoS guarantee in A-Tune-Online, our key insight is that **avoiding negative affect of historical observations with**

efficient optimization restart and improving precision of explicit workload shift detection is necessary. Precision in workload shift detection is particularly crucial in online tuning for two reasons. First, falsely identifying a workload shift without actual change can result in wasted historical observations and degraded quality of configuration recommendations. Second, workloads with minor differences may exhibit correlated performance across configurations, as illustrated in Fig. 11(a), suggesting that accurately retaining prior tuning experience can benefit current workloads.

While the detection-guided re-optimization avoids being misled by previous observations, and thus plays a significant role in the efficiency and performance of online tuning, how to effectively re-optimize poses unique challenges. This is due to the fact that with limited observations during early re-optimization iterations, the surrogate model fitting might become unreliable. Moreover, BO may potentially select bad configurations during the tuning process because, without constraints, the EI acquisition function would explore regions with substantial uncertainty. As a result, blindly restarting optimization can degrade the best-tuned performance, slow down tuning convergence, and violate QoS guarantee due to unconstrained configurations exploration. Specifically, we encounter the following challenges.

Challenge 1: It is challenging to maximize the precision of detection while maintaining a high recall. It is easy to improve detection recall by designing a lenient detection criterion frequently triggering the workload shift signals, but it significantly reduces the detection precision, thereby reducing the tuning stability. Alternatively, combining multiple indicators can mitigate shortcomings of a single indicator detection. However, blind combination of multiple indicators does not necessarily improve precision and recall, and we need rigorous theoretical proof to decide how to choose and combine multiple indicators.

Challenge 2: It is challenging to ensure efficiency and performance of re-optimization with limited initial observations. In the initial phase of restarting the optimization, simply relying on BO is insufficient to obtain guaranteed tuning efficiency and performance, because re-training surrogate model from scratch with limited observations. Hence, it becomes necessary to consider how to transfer knowledge from historical task experiences for warm-starting, while minimizing the potential negative impact of historical observations.

Challenge 3: It is challenging to ensure QoS of optimization with poor sampling. Most optimization algorithms inherently exhibit characteristics such as broad selection regions and high uncertainty. For example, BO strategically balances exploitation, which focuses on areas known to yield good results, and exploration, which targets less understood regions to discover potentially better solutions. The exploration can lead to broad selection regions due to the probabilistic estimation of the objective function across the search space, particularly in areas of high model uncertainty. Consequently, predicting whether the next recommendation will degrade performance transiently and violate service QoS is inherently

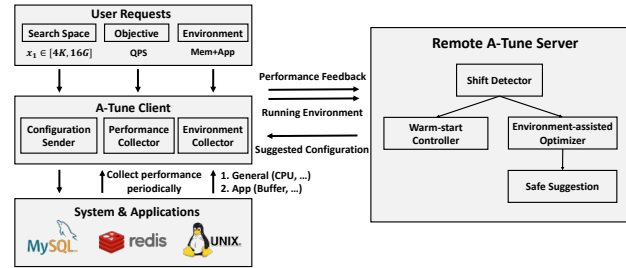


Fig. 1: System overview of A-Tune-Online.

difficult but essential for online tuning.

C. A-Tune-Online Architecture

Architecture Overview. Fig. 1 presents the system architecture of A-Tune-Online. The **A-Tune client** is responsible for orchestrating the entire configuration tuning process. The **A-Tune server** stores the history of all tuning tasks and continuously suggests configurations. To define a tuning task, end users first need to specify a search space, an objective, an interval for tuning, and running environments to be monitored (optional). Then, the remote A-Tune server creates a new tuning task and optimizes the objective iteratively. During each iteration, the A-Tune client collects the performance, samples the running environments of the database, and sends that information to the A-Tune server. The A-Tune server stores the information as the tuning history and suggests the most promising configuration based on the history. Finally, the A-Tune client deploys the new configuration.

Key design 1: the multi-indicator multi-stage detector (§IV-B). A-Tune-Online integrates a multi-indicator rule-based and replay-based detector to construct a multi-stage workload shift detection strategy for high precision and recall.

Key design 2: similarity-based dual warm start scheme (§IV-C). A-Tune-Online restarts the optimization when detecting a workload shift. It uses an ML-based predictor to indicate the similarity between the current workload and previous ones. To improve the efficiency and performance of re-optimization while avoiding the negative impact of dissimilar historical tasks, it employs two levels of warm-starting based on the degree of similarity.

Key design 3: lower confidence bound enhanced QoS-aware BO (§IV-D). To avoid the risk of performance degradation from BO exploring highly uncertain areas, A-Tune-Online builds a safety region based on the lower confidence bound and a QoS guarantee, and restricted sampling to this area.

IV. ONLINE CONFIGURATION TUNING

A. Workflow Overview

Fig. 2 shows the A-Tune-Online server workflow. In each iteration, the server receives performance feedback and current running environments from the client. The information is stored in the **history database**. The **shift detector** decides whether the current workload significantly shifts (i.e., is quite different from the previous iterations of the same tuning task).

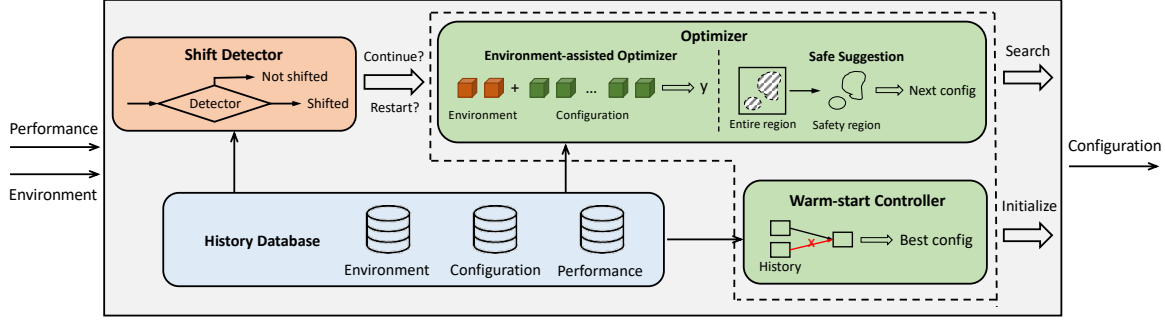


Fig. 2: Workflow of A-Tune-Online server.

If a shift is detected, the optimal configuration for the previous workload may no longer be suitable, and re-optimization is needed. Utilizing the stored tuning history, the **warm-start controller** selects the most similar tasks for a rapid warm-start. If no shift is detected, an environment-assisted **optimizer** is constructed based on current workload observations, and new configurations are suggested via safe recommendation.

Environment Vectors. A-Tune-Online characterizes workloads using environment vectors [9] with both general and application-specific types. General vectors refer to common metrics (such as average CPU utilization and memory usage) collected by the operating system in various scenarios. Application-specific vectors refer to metrics collected from the specific application. A-Tune-Online collects metrics reported in logging files, which has two advantages: 1) Logging files are readily accessible and contain comprehensive running information. 2) Metrics within logging files (e.g., Spark executor execution time) are more secure than high-level information (e.g., the distribution of input data) which may be unavailable due to privacy concerns.

A-Tune-Online further leverages environment vectors to detect shifts and learn the similarity between history tasks. Moreover, to enable BO to adapt to both significant and slight workload changes, A-Tune-Online applies the environment-assisted surrogate, which takes the environment along with configuration into consideration while modeling the objective value with the Probabilistic Random Forest.

B. Multi-stage Shift Detection

Unlike offline tuning with a static workload, online tuning deals with a dynamic workload that shifts over time. Therefore, accurate workload shift detection is essential to ensure the efficiency, performance, and stability of online tuning.

Multi-stage detection strategy. As shown in Fig. 3, in A-Tune-Online, we design a multi-stage shift detector to ensure both high precision and recall rate. In the first stage, A-Tune-Online employs a multi-indicator precision-first detector, using the intersection of multiple rule-based conditions to ensure a conservative shift detection. In other words, this design aims to reduce the false positive rate, as the application performance may be unstable in early iterations if we restart optimization on the workload. Although using a consensus of

multiple indicators for shift detection can improve precision, it reduces recall by decreasing the frequency of shift signals. Therefore, we proceed to the next stage only when the multi-indicator rule-based detector fails to identify a workload shift but some of the indicators suggest a workload shift. This stage conducts a more costly but accurate detection using replay-based detector. Once a workload shift is detected at any stage, the server will start a new tuning process on the current workload and stop the previous one.

Multi-indicator rule-base detector. We first use predefined rules to determine if a workload shift has occurred. To avoid frequently restarting the tuning process when no shift has actually happened, A-Tune-Online relies on multiple indicators to deliver highly precise detection. A workload shift signal is only triggered when all indicators agree that a shift has occurred. Below, Theorem 2 presents the precision lower bound for shift detection judgment that integrates multiple indicators in a consistent manner. Subsequently, Corollary 1 and 2 illustrate what kind of indicators to combine can enhance detection precision. Furthermore, we discuss the feasibility of identifying such indicators in online tuning and detail the specific ones employed by A-Tune-Online.

Theorem 1. *Let there be n indicators $A_1, A_2, \dots, A_n$. Each has recall $R_i = P(A_i = 1 \mid C = 1)$. Here, $C = 1$ represents that a workload shift has actually occurred, and $A_i = 1$ indicates that the i -th indicator predicts a workload shift. The recall when all indicators simultaneously predict 1 is:*

$$P\left(\bigcap_{i=1}^n A_i = 1 \mid C = 1\right) \geq \sum_{i=1}^n R_i - (n - 1). \quad (2)$$

Proof. We use induction on the number of events n .

Base case ($n = 2$): Using the inclusion-exclusion principle:

$$P(A_1 = 1, A_2 = 1 \mid C = 1) \geq R_1 + R_2 - 1.$$

Inductive step: Assume the statement holds for $n - 1$. For n events, consider

$$\begin{aligned} P\left(\bigcap_{i=1}^n A_i = 1 \mid C = 1\right) &= P\left(A_n = 1, \bigcap_{i=1}^{n-1} A_i = 1 \mid C = 1\right) \\ &\geq R_n + \left(\sum_{i=1}^{n-1} R_i - (n - 2)\right) - 1 = \sum_{i=1}^n R_i - (n - 1). \end{aligned}$$

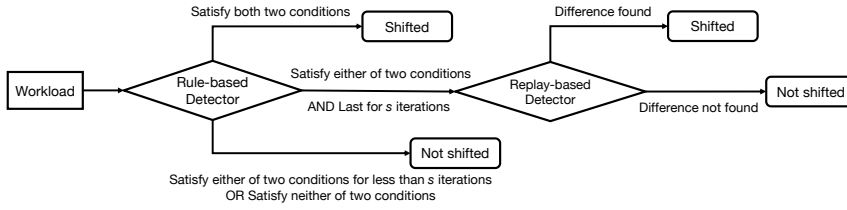


Fig. 3: The flowchart of A-Tune-Online shift detector.

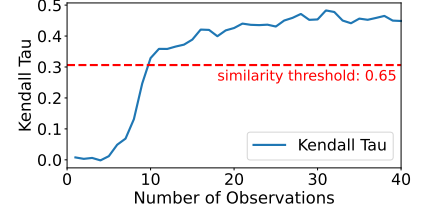


Fig. 4: Kendall-tau correlation coefficient between surrogate model predictions and true performance on random configurations.

This completes the proof by induction. $\square$

Theorem 2. Let there be n independent indicators $A_1, A_2, \dots, A_n$. Each indicator has precision $P_i = P(C = 1 | A_i = 1)$ and recall $R_i = P(A_i = 1 | C = 1)$. Assuming independence among the indicators, the precision when all indicators simultaneously predict 1 is given by:

$$P(C = 1 | \mathbf{A} = \mathbf{1}) \geq \frac{\sum_{i=1}^n R_i - (n-1)}{\prod_{i=1}^n R_i} \cdot \frac{\prod_{i=1}^n P_i}{P^{n-1}(C = 1)} \quad (3)$$

where $\mathbf{A} = (A_1, A_2, \dots, A_n)$ is the vector of all indicators.

Proof. To find the precision $P(C = 1 | \mathbf{A} = \mathbf{1})$, we start by using the definition of conditional probability:

$$P(C = 1 | \mathbf{A} = \mathbf{1}) = \frac{P(\mathbf{A} = \mathbf{1} | C = 1) P(C = 1)}{P(\mathbf{A} = \mathbf{1})} \quad (4)$$

Using the independence of indicators, we have:

$$P(\mathbf{A} = \mathbf{1}) = \prod_{i=1}^n P(A_i = 1) \quad (5)$$

With the definition of precision and recall for each indicator:

$$P(C = 1, A_i = 1) = P_i \cdot P(A_i = 1) = R_i \cdot P(C = 1)$$

From this, we can derive:

$$P(A_i = 1) = \frac{R_i \cdot P(C = 1)}{P_i} \quad (6)$$

Finally, the precision is:

$$P(C = 1 | \mathbf{A} = \mathbf{1}) = \frac{P(\mathbf{A} = \mathbf{1} | C = 1)}{\prod_{i=1}^n R_i} \cdot \frac{\prod_{i=1}^n P_i}{P^{n-1}(C = 1)}$$

According to Theorem 1, the proof completes. $\square$

With Theorem 2, we can make the following corollaries:

Corollary 1. Suppose we have $n(n \geq 2)$ independent indicators, each with recall greater than $n/(n+1)$ and precision greater than $P(C = 1) \cdot n^{\frac{n}{n-1}}/(n+1)$. In that case, the precision of predicting a shift when all indicators simultaneously agree is greater than the precision of any single indicator.

Proof. With Theorem 2, we have

$$P(C = 1 | \mathbf{A} = \mathbf{1}) \geq \frac{\sum_{i=1}^n R_i - (n-1)}{\prod_{i=1}^n R_i} \cdot \frac{\prod_{i=1}^n P_i}{P^{n-1}(C = 1)}$$

Because for any R_i , the left-hand side of the above expression monotonically increases from $\frac{n}{n+1}$ to 1, therefore:

$$P(C = 1 | \mathbf{A} = \mathbf{1}) > \frac{(n+1)^{n-1}}{n^n} \cdot \frac{\prod_{i=1}^n P_i}{P^{n-1}(C = 1)} \quad (7)$$

For any $j \in \{1, 2, \dots, n\}$, we compare

$$\frac{P(C = 1 | \mathbf{A} = \mathbf{1})}{P_j} > \frac{(n+1)^{n-1}}{n^n} \cdot \frac{n^n}{(n+1)^{n-1}} = 1$$

This completes the proof. $\square$

Corollary 2. Suppose we have $n(n \geq 2)$ independent indicators, each with a recall greater than $n/(n+1)$ and a precision greater than $(n^2/(n^2-1))^{n-1} P(C = 1)$ for predicting a workload shift. In that case, the precision of predicting a shift when all indicators simultaneously agree has higher infimum than the precision when any subset of size $n-1$ agrees.

Proof. First, with (7), we have the lower bound of the simultaneous prediction precision,

$$l_{P(C=1|\mathbf{A}=\mathbf{1})} = \frac{(n+1)^{n-1}}{n^n} \cdot \frac{\prod_{i=1}^n P_i}{P^{n-1}(C = 1)}$$

For any subset $S \subseteq \{1, 2, \dots, n\}$ with $|S| = n-1$, We compare:

$$\begin{aligned} \frac{l_{P(C=1|\mathbf{A}=\mathbf{1})}}{l_{P(C=1|\mathbf{A}_S=\mathbf{1})}} &= \left(\frac{n^2-1}{n^2}\right)^{n-1} \cdot \frac{\prod_{i=1}^n P_i \cdot P^{n-2}(C = 1)}{\prod_{i \in S} P_i \cdot P^{n-1}(C = 1)} \\ &= \left(\frac{n^2-1}{n^2}\right)^{n-1} \cdot \frac{P_j}{P(C = 1)} > 1 \end{aligned}$$

where $j \notin S$. This completes the proof. $\square$

Based on the real-world workload characteristic described in §II-A, the workload remains similar for a period before shifting. Consequently, the number of workload shifts relative to the detection iteration is typically small (for example, during the execution of a workload, dozens of tuning and detections might occur, with $P(C = 1)$ being around 0.01 or lower). Thus, it is feasible to find individual indicators with precision much greater than $P(C = 1)$. We can also achieve a recall greater than $n/(n+1)$ by appropriately relaxing the criteria. According to Corollary 1, combining these indicators can enhance detection precision. Furthermore, Corollary 2 suggests that adding more indicators theoretically leads to precision converging towards an optimal direction.

A-Tune-Online applies manual conditions to detect workload changes based on environment vectors. The detector monitors environments using a sliding window S of length s , which means we store environment vectors of s previous configurations. Concretely, at the t^{th} iteration, A-Tune-Online use the following two indicators.

- By calculating the variance for each dimension in $S_{[t-s, t-1]}$, the variance in any β dimensions exceeds θ_v .
- By calculating the difference in the mean for each dimension between two windows $S_{[t-2s, t-s-1]}$ and $S_{[t-s, t-1]}$, the difference in any β dimensions exceeds θ_m .

As $n = 2$, We need the precision of the indicator greater than $4/3 \cdot P(C = 1)$ and recall greater than $2/3$. In practice, we set s as 5, β as 40% of all dimensions. Both θ_m and θ_v are set to 0.4. Our tests show that this parameter setting satisfies the precision and recall constraints, achieving good combined performance with generality across the five selected scenarios. Additionally, §V-D1 provides a sensitivity analysis of the parameter settings. If both two conditions are met, we consider the workload is shifted and restart optimization. If neither condition is met, the workload is not considered shifted. If only one condition is met in s consecutive rounds, we proceed to the next stage using the replay-based detector.

Replay-based detector. In A-Tune-Online, we adopt the replay-based detector in the second stage, which monitors the difference in environments by re-evaluating a configuration that has been deployed before. To prevent performance from degrading due to replaying poor configurations, we only replay the best configuration found $2s$ iterations before (skip $2s$ configurations already used). The condition is as follows,

- By comparing the two environment vectors, the difference of any β dimensions is over θ_m .

If the condition is met, we consider the workload as a shifted one, or else not. Since replaying a configuration is of high cost (i.e., it will take a tuning interval to replay, during which no new configuration can be suggested), the replay-based detector can be applied again after at least s iterations.

C. Re-optimization with Warm Start

Upon detecting a workload shift, A-Tune-Online starts a new tuning task on the current workload and isolates it from historical observations. However, optimization restart requires re-training surrogate model from scratch, leading to extra performance overhead. Given that the remote server handles multiple user requests concurrently, each potentially involving multiple tuning tasks due to workload shifts, the history database accumulates extensive tuning history. Each task's history includes multi-round configuration performance observations for a specific workload. Inspired by transfer learning [2], [19], [20], the history of different tuning tasks in the same online tuning process can improve efficiency during early re-optimization iterations following a workload shift. Thus, A-Tune-Online must rapidly select appropriate similar history tasks to ensure safe transfer.

Leveraging history tasks can enhance efficiency but also introduces the risk of negative impacts from historical observations. Therefore, A-Tune-Online employs a warm-start strategy, utilizing historical task observations only during the initial phase of optimization. On the one hand, if similar historical tasks are successfully identified, we aim to allow tasks with higher similarity to exert a greater positive impact. On the other hand, if no historical tasks are similar to the current task, or if dissimilar tasks are mistakenly selected as similar, transferring from those tasks may fail to improve efficiency. However, after the warm-start phase, the system relies solely on observations from the current task to recommend configurations, avoiding persistent impact from observations of dissimilar tasks. In the worst case, the system efficiency matches that of vanilla BO. Overall, A-Tune-Online effectively leverages the positive influence of highly similar tasks while minimizing the negative effects of dissimilar tasks.

Concretely, A-Tune-Online extends Rover [2] for task selection and knowledge transfer. We utilize environment vectors as meta-features for each scenario. A regression model, which takes two meta-features as inputs, is learned to predict the pair-wise similarity between tuning tasks. The training data is derived from the tuning history in the history database. The similarity is defined as the ratio of concordant pairs of surrogate predictions on random configurations. A high similarity indicates that the ranks of the same configuration are similar, implying that the optimal configuration may be similar in both tasks. We employ LightGBM as the regressor for modeling similarity.

After training the regression model, when a workload shift is detected, we compute its similarity to all history tasks using environment vectors. Based on predicted similarity, A-Tune-Online performs two levels of history knowledge reuse.

- 1) **Warm start with configurations.** We select the top five tasks with the largest similarity, and initialize the new tuning task with the best configurations of similar tasks.
- 2) **Warm start with surrogates.** if the largest similarity exceeds θ_s , we use the surrogate model from the most similar task to generate configurations during the first T iterations. As shown in Fig. 4, we apply BO to 50 benchmarks extracted from five scenarios. In each round, we record the average Kendall-tau correlation between the surrogate model's predictions and the true performance of randomly sampled configurations. The red dashed line shows the average Kendall-tau correlation using predictions from surrogate models of similar tasks with similarity scores above 0.65. The surrogate model's fitting ability for the current task improves significantly after about 10 iterations, while before 10 iterations, its performance is inferior to that of surrogate models from similar tasks with scores above 0.65. Therefore, we set $\theta_s = 0.65$ and $T = 10$. If training samples are insufficient for the regressor, we use Euclidean distance to choose similar tasks and disable the reuse of the surrogate model.

Algorithm 1: Pseudo-code of A-Tune-Online tuning iteration.

Input: the tuning interval P , the search budget for each task $\mathcal{B}$, the configuration search space $\mathcal{X}$, the safety threshold δ .

```
1: while  $P$  is reached do
2:   Collect the environment vectors  $e_i$ .
3:   Collect the performance  $y_i$  of the previous configuration  $x_i$ .
4:   if  $|D| < \mathcal{B}$  then
5:     Augment the observations  $D = D \cup (x_i, e_{i-1}, y_i)$ .
6:   end if
7:   if a shift is detected then
8:     Reset observations  $D = \emptyset$ . // Restart optimization.
9:   end if
10:  if  $|D| < 5$  then
11:    return the best configurations from similar history tasks.
12:  else if  $|D| < \mathcal{B}$  then
13:    if  $|D| < 10$  and a similar task is found then
14:      Reuse the surrogate  $M$  from the most similar task.
15:    else
16:      Train a surrogate  $M$  on current observations  $D$ .
17:    end if
18:    Sample configuration candidates from  $\mathcal{X}$  and
    maintain a safety region  $\mathcal{X}_{safe}$  based on  $\delta$ .
19:    Choose the next configuration by
     $x_{i+1} = \arg \max_{x \in \mathcal{X}_{safe}} \alpha(x, e_i; M)$ .
20:    return the next configuration  $x_{i+1}$ .
21:  else
22:    return the optimal configuration  $x^*$  found in  $D$ .
23:  end if
24: end while
```

D. QoS-aware Suggestion

Offline tuning deploys the best configuration after sufficient evaluations on a static workload copy, whereas online tuning directly evaluates all configurations using the real workload. Consequently, evaluating poor configurations in online tuning inevitably degrades performance and violates customer QoS guarantees. As described in §II-C, BO selects configurations with the highest expected improvement, but due to high predictive variance, these configurations may perform poorly. To preserve the QoS of the proposed configuration, we further propose safe suggestion to select the next candidate.

Given the predictive mean $\mu_t(x)$ and variance $\sigma_t^2(x)$ by the BO surrogate built at the t^{th} iteration, we use the predictive lower bound to select safe configurations. The lower bound of a given configuration x is defined as:

$$l_t(x) = \mu_t(x) - \eta \sigma_t(x) \quad (8)$$

where η is a constant that controls the strictness of QoS guarantees by determining the expected probability that a configuration's actual performance will exceed its lower confidence bound. Statistically, the probability is 68.3% when $\eta = 1$ and increases to 95.5% when $\eta = 2$. A small η reduces suggestions safety, while a high η hampers search space exploration. We follow default settings $\eta = 1$ in algorithms with confidence bound (e.g., UCB acquisition function) to balance safety and exploration. However, in production environments with higher safety requirements, a larger η can be set to prioritize safety.

During each iteration, we build a safety region $\mathcal{X}_{safe} = \{x \in \mathcal{X} | l_t(x) \geq \delta\}$, where the predictive lower bounds of those configurations' performance are expected to exceed a

given threshold δ . The safety region tends to expand as more observations are accumulated across the space, helping us to avoid blindly exploring high-uncertainty or poor-performance regions. δ represents the QoS requirement, and the current A-Tune-Online implementation sets it to the performance of the default configuration in the current task, which can be easily acquired with historical knowledge or as an initial configuration of optimization. Given a set of candidate configurations, A-Tune-Online only suggests the candidate with the largest expected improvement value within the safety region.

E. Algorithm Summary

Algorithm 1 shows the pseudo-code of each online tuning iteration. Upon reaching the tuning interval, the server receives environment vectors and performance feedback from the client (Lines 2-3), and stores the evaluated configuration information in the history database (Line 5). The detector assesses workload shifts and restarts the optimization upon detection (Lines 7-8). The optimization initialization involves applying the best configurations from the top-5 similar historical tasks (Line 11). If sufficient similarity is found, the surrogate model from the most similar task is reused in early iterations (Lines 13-14). After warm-start, A-Tune-Online trains an environment-assisted surrogate and selects the next configuration based on the safety region and acquisition function (Lines 16-20). After exhausting the search budget for the current tuning task, A-Tune-Online returns the best configuration found in the observations (Line 22), and stops suggesting new configurations until a workload shift is detected.

V. EXPERIMENTS

A. Experiment Setups

Scenarios. To assess the effectiveness of A-Tune-Online, we evaluate it across five scenarios: MySQL, Redis, Kafka, UnixBench, and Spark on Hadoop. To mimic real-world online workload, we utilize public benchmarks and periodically switch between workloads. Concretely, the benchmarks and settings for each scenario are detailed as follows.

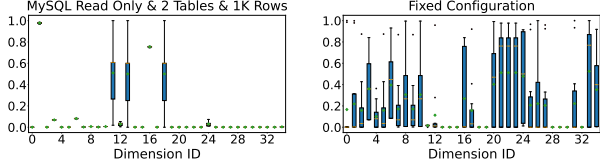
MySQL. We use the OLTP workloads from Sysbench [21], including three types: "Read only" (RO), "Write only" (WO), and "Read & Write" (RW). Each workload comprises 2 or 8 tables with 1k or 100k rows, resulting in 12 workloads.

Redis. We use the built-in Redis-benchmark, including four workload types: "set", "get", "sadd", "mset". The number of parallel connections is 10, 100, or 1000, and the data size is 1, 5, or 10 bytes, resulting in a total of 36 different workloads.

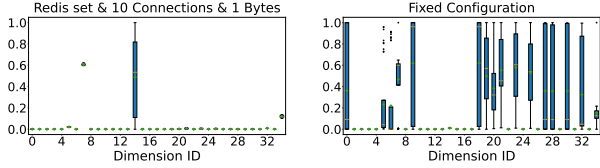
UnixBench. We select 12 different workloads that test different aspects of the operating system from UnixBench, which are "dhry2reg", "whets", "syscall", "pipe", "context1", "spawn", "execl", "fstime-w", "fstime-r", "fsdisk-w", "fsdisk-r", "hanoi".

Kafka. We use the built-in benchmark tool to generate the workload. Specifically, the number of partitions is 1, 5, or 10. The record size is 256, 1024, or 4096 bytes. There are 9 different workloads in total.

Spark. We use 24 out of 29 workloads from Hibenach [22]



(a) Given different configurations. (b) Given different workloads.
Fig. 5: Distribution of environment vectors on MySQL.



(a) Given different configurations. (b) Given different workloads.
Fig. 6: Distribution of environment vectors on Redis.

We provide the search space of each scenario, which are 13-d, 13-d, 39-d, 8-d, and 9-d for MySQL, Redis, UnixBench, Kafka, and Spark, respectively. We tune dynamic configuration knobs without restarting the service, as restarts are unacceptable in an online tuning system. More details are available in the repository <https://github.com/PKU-DAIR/A-Tune-Online>.

Environments. A-Tune-Online collects application logs to construct environment vectors. For each scenario, we obtain the specific vector as follows,

- MySQL: a 67-d vector from the table “information_schema.INNODB_METRICS” in MySQL.
- Redis: a 68-d vector using the built-in function `info()` in the Python package Redis.
- UnixBench: an 18-d vector by commands “`vmstat`”.
- Kafka: a 10-d vector with the built-in JMXTool.
- Spark: a 24-d vector from the file Spark-EventLog.

Baselines. We compare A-Tune-Online with the following state-of-the-art tuning algorithms: 1) GP-BO [1], [2], [5]: A BO method that uses the Gaussian process as the surrogate and EI as the acquisition function. 2) TPE [3], [15]: A BO method that applies the tree-structured parzen estimators as the surrogate. 3) OnlineTune [9]: A BO-based online tuning method that clusters historical observations into distinct groups, then trains an individual surrogate model for each cluster. 4) DDPG+ [7]: reinforcement learning (RL) that improves DDPG [23] by using evolutionary algorithm for initialization; 5) QTune [6]: RL with DS-DDPG. As some of the previous work focus on offline performance tuning, we change their settings and apply them directly to online tuning.

Experiment Settings. To measure the tuning performance of different methods, we tune the KB per second (KBps) for IO tasks in UnixBench, loops per second (lps) for the other tasks in UnixBench, opposite of runtime for Spark, request per second (rps) for MySQL, Redis, and Kafka. During optimization, we collect the performance and environment vectors every 10 minutes and then suggest a new configuration. If the workload does not shift, a tuning task will finish if 40 configurations have been evaluated. And then, the best observed configuration is deployed until a shift is detected.

Running Environment. All the experiments are conducted using a machine with 96 CPU cores and 100G memory. A-Tune-Online implements Bayesian optimization based on OpenBox [24], a toolkit for black-box optimization. The operating system is OpenEuler 22.03 [25], and application versions are MySQL 8.0.29, Redis 6.0.5, Kafka 3.5.0, UnixBench 5.1.3, Hadoop 3.2.3, and Spark 3.2.4.

B. Analysis of Environment Vectors

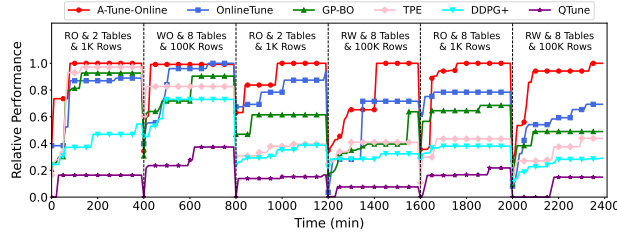
We conduct an experiment to investigate the ability of environment vectors in A-Tune-Online to identify workload changes. Fig. 5 and Fig. 6 depict the box plots of environment vectors in MySQL and Redis, respectively. Due to the space limit, we present half of the dimensions in the environment vector. Each left sub-figure shows environment vectors when evaluating ten randomly sampled configurations using the same workload, while right sub-figures, present for the same configuration with different workloads. The left sub-figures indicate stable environment vectors, with only three dimensions exhibiting significant variance in Fig. 5(a). Conversely, the right sub-figures show significant variation when workloads differ, with over 50% of dimensions with significant variance in Fig. 5(b). Therefore, we conclude that the environment vectors in A-Tune-Online are sensitive to workload changes instead of configuration changes. As described in §IV-B, A-Tune-Online decides workload shifts based on the difference in environment vectors. Utilizing thresholds in the detection strategy, significant changes in the environment vectors indicate underlying workload is more likely to shift. §V-D1 provides more evaluations of the detection strategy.

C. Comparison with Baselines

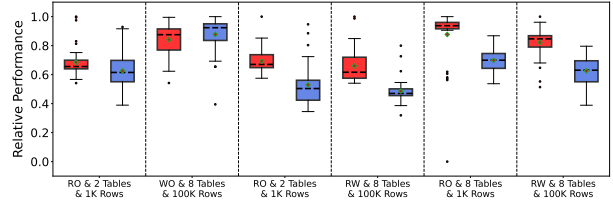
This section compares A-Tune-Online with state-of-the-art tuning algorithms. To simulate dynamic workloads, we utilize a combination of individual workloads sampled from benchmarks in each scenario, with each workload running for approximately 400 minutes before switching to another. We observe that application performance highly depends on the underlying workloads. For example, the value of RPS for databases is quite different when running read-only and write-only workloads. To ensure clear comparisons, we report the relative best-observed performance. We normalize performance for each workload by setting the best and worst values among all compared methods to 1 and 0, respectively. Additionally, we conduct experiments under two settings: starting tuning with and without history knowledge, to compare tuning algorithms under different circumstances.

Without history knowledge. In this setting, all tuning algorithms begin from scratch with an empty history database. Upon detecting a workload shift or reaching a termination limit, we store the current task’s observations as a new tuning task history in the database for subsequent transfer learning. The experiment is conducted on MySQL and Redis, with six individual workloads combined to form a dynamic workload.

Fig. 7(a) and Fig. 8(a) present the online tuning performance of the compared baselines. In the first workload, BO-

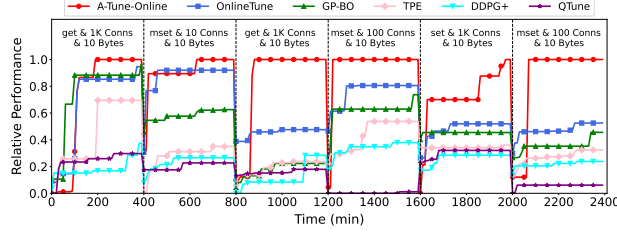


(a) Relative tuning performance on six individual workloads.

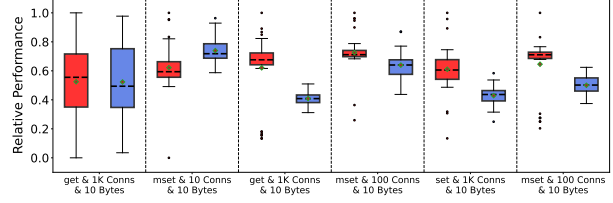


(b) Performance distribution of configurations during optimization.

Fig. 7: Online tuning performance without history knowledge on MySQL.

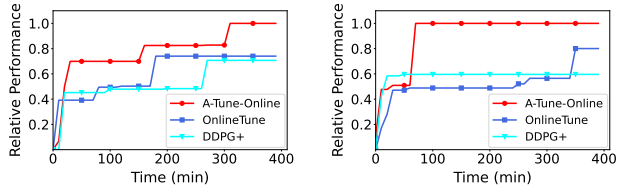


(a) Relative tuning performance on six individual workloads.

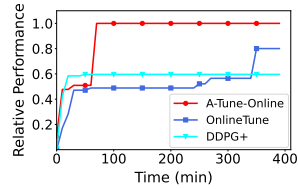


(b) Performance distribution of configurations during optimization.

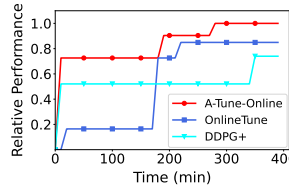
Fig. 8: Online tuning performance without history knowledge on Redis.



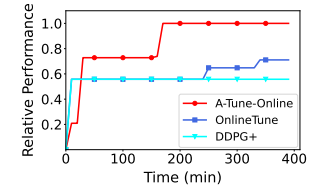
(a) MySQL RO & 2 Tables & 1K Rows.



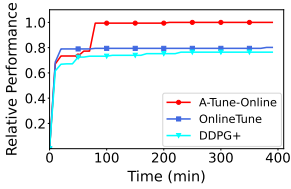
(b) Redis Mset & 1K Conns & 10 Bytes.



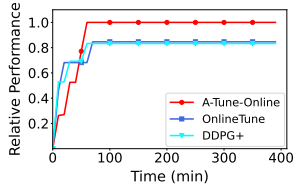
(c) Kafka 1 Partitions & 256 Bytes.



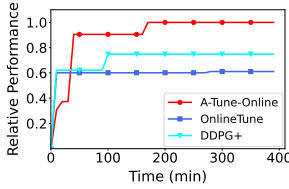
(d) Kafka 10 Partitions & 256 Bytes.



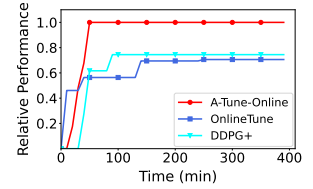
(e) Unix Fstime-w.



(f) Unix Fsdisk-w.



(g) Spark Sort.



(h) Spark Terasort.

Fig. 9: Online tuning performance with history knowledge on five scenarios. Each caption refers to a specific workload.

based methods outperform DDPG+ and QTune, aligning with previous findings [16] that RL performs worse than expected due to more required initial configurations. Among BO-based methods, performance is similar due to the absence of history knowledge. As time progresses, OnlineTune and A-Tune-Online gradually surpass other methods, as both of them handle dynamic workloads and leverage historical knowledge. However, OnlineTune cannot mitigate the negative effects of dissimilar workloads due to the instability of unsupervised clustering. For example, in MySQL and Redis, the mean mutual information between OnlineTune's classification results and actual workload labels of observation at each tuning is 0.59 and 0.57, respectively. In contrast, A-Tune-Online achieves higher values of 0.88 and 0.81 after shift detection and similar task matching, confirming its effectiveness in segmenting and utilizing historical observations. From another

perspective, in each tuning round, A-Tune-Online utilizes 97% and 96% of historical observations that belong to the same workload as the current task in MySQL and Redis, respectively, outperforming OnlineTune's 88% and 91%. Therefore, A-Tune-Online minimizes the risk of being misled by previous observations during workload shifts. Furthermore, it rapidly adapts to similar workloads and achieves better application performance by continuous shift detection and warm-start with similar historical tasks. DDPG+ and QTune exhibit suboptimal performance in later workloads, as RL models require diverse workloads for training but environment vectors are stable in the same individual workload as shown in §V-B.

Besides the best tuned performance in each workload, we also present the performance distribution across all suggested configurations during optimization in Fig. 7(b) and Fig. 8(b). Due to limited space, we only provide the results of A-Tune-

TABLE II: The percentage of cases that use different number of iterations for detection. ‘F’ refers to failure on detection.

	1	2	3	4	5	6	7	8	9	10	F
MySQL	35.6%	4.5%	7.6%	3.8%	12.1%	4.5%	1.5%	3.0%	1.5%	1.5%	24.2%
Redis	37.8%	12.5%	10.0%	3.7%	8.3%	6.7%	0.7%	0.6%	0.7%	8.6%	10.4%
Unix	46.2%	28.8%	12.1%	0.0%	3.0%	0.0%	0.0%	0.0%	0.8%	0.0%	9.1%
Kafka	55.6%	19.4%	2.8%	5.6%	0.0%	0.0%	0.0%	0.0%	2.8%	0.0%	13.9%
Spark	41.1%	25.2%	5.4%	5.3%	4.9%	1.4%	0.7%	0.0%	1.4%	2.7%	11.8%

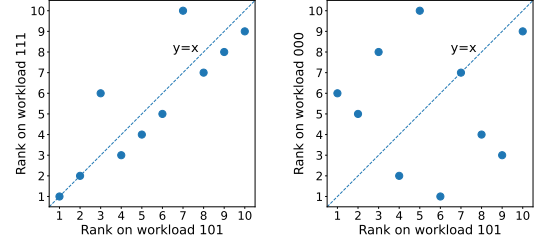
	000	001	010	011	100	101	110	111	200	201	210	211	
First digit:													
0 - Write only	000	-	3	-	3	5	1	5	1	-	3	-	2
1 - Read only	001	6	-	8	-	5	5	5	3	5	-	-	-
2 - Read & Write	010	-	4	-	3	5	1	5	1	-	9	-	4
	011	6	-	8	-	5	5	3	4	6	-	7	-
Second digit:													
0 - Two tables	100	1	1	1	1	-	1	-	9	1	1	1	1
1 - Eight tables	101	1	2	1	1	10	-	7	-	1	1	1	1
	110	1	1	3	2	-	-	-	8	1	1	1	1
Third digit:													
0 - 1K rows	111	1	2	2	3	8	-	10	-	1	1	1	1
1 - 100K rows	200	-	4	-	5	1	1	1	1	-	4	-	3
e.g., "001" represents a	201	5	-	-	-	1	1	1	1	5	-	6	-
Write only task with two	210	-	-	-	6	1	1	1	1	-	5	-	3
tables and 100K rows.	211	3	-	5	-	1	1	1	1	4	-	2	-

Fig. 10: Detailed analysis on the detection strategy on MySQL. Each value in the table refers to the number of iterations required to identify a shift from the workload in the row to the column. For example, the value “4” in row “200” and column “001” indicates that it takes 4 iterations to detect a shift from a Read & Write task with Two tables and 1K rows to a Write only task with Two tables and 100K rows.

Online and OnlineTune, which achieve the best performance. A-Tune-Online realizes a higher mean and thus safer configurations than OnlineTune thanks to the better reliability of the workload shift detector compared to unsupervised clustering.

With history knowledge. This setting evaluates the ability of each method to leverage previous tuning history knowledge on previous workloads. We prepare this history by applying vanilla BO to each workload detailed in §V-A. GP-BO and TPE are excluded due to their inability to distinguish between workloads, as demonstrated in previous experiments. Fig. 7 shows the results of A-Tune-Online, OnlineTune, and DDPG+. The experiment is conducted on all 5 scenarios, with all methods using the same tuning history. We normalize the performance to the default configuration, which is set as 0. We use the tuning history in a leave-one-out manner that only uses the history of the remaining workloads. For instance, when tuning one of the 12 MySQL workload, the history of the other 11 workloads is used.

Fig. 9 presents a part of the tuning results in five scenarios. A-Tune-Online identifies optimal configurations more rapidly and converges better than OnlineTune and DDPG+. Overall, A-Tune-Online improves the best-tuned performance by **2.86%** to **13.20%** and **4.68%** to **13.18%** over OnlineTune and DDPG+, respectively. Notably, on Spark, A-Tune-Online finds a near-optimal configuration within five iterations, suggesting that tuning history from similar tasks is valuable in online tuning. When achieving the same tuned performance, A-Tune-Online achieves average speedups of **2.90x** and **1.72x** over OnlineTune and DDPG+, respectively, with median speedups of **2.74x** and **1.66x**. In summary, A-Tune-Online more effectively leverages historical knowledge and adapts faster to workload shifts than existing approaches.



(a) Similar workloads.

(b) Dissimilar workloads.

Fig. 11: Correlation of configuration performance between similar and dissimilar workloads.

D. Algorithm Analysis

1) Analysis of Shift Detection: This section evaluates the shift detection strategy in A-Tune-Online. We switch workloads in pairs and report the number of configurations required to identify the workload shift. For instance, MySQL has 12 workloads, leading to $12 \times 11 = 132$ cases. Table II presents the results of five scenarios. The recall rate of the strategy ranges from 76% to 91%. Additionally, the replay-based detector improves the recall by an average of 8%. On average, each shift experiment triggers 0.6 replays, resulting in a cost of 6 minutes given our setup. Notably, most shifts are detected with a few evaluations (≤ 5). Because of the strategy’s conservative design, the precision rate is 100% in all scenarios.

Detection Details. Fig. 10 provides a detailed analysis of all 132 MySQL cases. Italic values indicate detections by the replay-based detector. The overall recall rate on is 76%. When there is a significant difference between read-only and write-only workloads, our strategy identifies shifts within 5 iterations. However, the strategy fails to detect insignificant workload difference, such as an increase in table count from 2 to 8. We further analyze this in Fig. 11, comparing the performance ranks of 10 random configurations on two workloads (workload 101 vs. 111 and 000, as shown in Fig. 10). The Kendall-tau correlation coefficients in Fig. 11(a) and Fig. 11(b) are 0.73 and -0.02, respectively. This indicates a strong positive correlation between configuration performance on two workloads with similar environment vectors, suggesting that near-optimal configurations in one workload are also effective in another similar workload. Thus, failing to identify slight differences may not significantly affect online performance.

Sensitivity Analysis. We analyze MySQL tuning process. Table III shows the effect of window size s when $\beta = 0.4$, $\theta_m = \theta_v = 0.4$. ‘Delay’ refers to extra iterations for detection after workload shift. Larger s improves detection performance (precision and recall) but increases the cost (delay), leading us to select $s = 5$ for a moderate delay. Then Table IV shows the effects of β and $\theta = \theta_m = \theta_v$ when $s = 5$. Larger β or larger θ leads to higher precision but lower recall. As re-optimization is time-consuming, we prioritize 100% precision. Although some recall may be higher, slight workload changes have minor impact when configuration ranks are similar as explain in Fig. 11. Thus, we set $\beta = 0.4$, $\theta_m = \theta_v = 0.4$ for better precision and delay.

TABLE III: Sensitivity analysis of detection window size s .

	Recall	Precision	Delay
$s = 3$	69.70%	100%	1.89
$s = 5$	75.76%	100%	3.41
$s = 7$	78.79%	100%	4.90

TABLE IV: Sensitivity analysis of β and θ .

Precision/Recall/Delay	$\theta = 0.2$	$\theta = 0.4$	$\theta = 0.6$
$\beta = 0.3$	0.93 / 0.98 / 3.53	0.99 / 0.83 / 3.12	1.00 / 0.69 / 3.97
$\beta = 0.4$	0.93 / 0.95 / 3.51	1.00 / 0.76 / 3.41	1.00 / 0.55 / 3.48
$\beta = 0.5$	0.94 / 0.95 / 3.51	1.00 / 0.70 / 3.34	1.00 / 0.54 / 3.80

2) *Ablation Study of Warm-start*: This section conducts an ablation study on the warm-start controller in A-Tune-Online while conducting experiments with history knowledge. Table V presents the performance speedups on all five scenarios. The results confirm a significant improvement in tuned performance when warm-start is applied to shifted workloads. Warm-start achieves speedups ranging from of **1.31x** to **3.83x** over the case without the warm-start controller. This is because of the high cost of re-optimization on shifted workloads, as the surrogate requires several observations for initialization. The warm-start controller not only enhances online tuning performance in early iterations following workload shifts but also provides high-quality observations for surrogate training.

3) *Ablation Study of Safety Suggestion*: Table VI reports the frequency of each method violating the safety threshold. We first notice that in some scenarios like Unix, though different configurations lead to different performance, bad configurations may not cause significant performance degradation, where most baselines can suggest safe configurations. However, in other scenarios, there is a clear reduction of unsafe suggestions when using A-Tune-Online with safe suggestion. Remarkably on Kafka, A-Tune-Online reduces the average ratio of unsafe configurations by **2.23%** compared to Online-Tune with a worst-case reduction of **5%**.

4) *System Overhead*: Efficiently executing configuration recommendations is crucial in online tuning. We list the average time for each component to generate a configuration on MySQL with 12 history tasks and 13-d search space. Shift detection (rule-based detection) takes less than 0.1 seconds. For warm-start, similarity modeling takes 2.30 seconds, which can be ignored as the model can be trained in parallel during optimization. Configuration suggestion takes 0.1 seconds for warm-start configuration (to compute task similarity) or 3.32 seconds for safe suggestion (to train the surrogate model and sample configurations for a recommendation). To sum up, compared to the usual optimization intervals, the overhead of our system to recommend a configuration is minimal.

VI. RELATED WORK

To improve the software performance, a wide variety of configuration tuning methods have been proposed to tune parameters for a certain application, including databases [1], [23], [26], [27], big data analysis [2], [5], [28], network intensive applications [29], operating systems [30], etc.

Among the literature, most methods apply an offline tuning procedure. These methods optimize the application perfor-

TABLE V: Performance speedup of A-Tune-Online relative to A-Tune-Online without warm-start.

	MySQL	Redis	Kafka	Unix	Spark
Speedup	2.43x	3.83x	1.42x	1.31x	2.50x

TABLE VI: Ratio of configurations that violate the safety threshold. A-Tune-Online- refers to A-Tune-Online without safety suggestion.

	GP-BO		OnlineTune		A-Tune-Online-		A-Tune-Online	
	Avg	Max	Avg	Max	Avg	Max	Avg	Max
MySQL	13.75%	75.00%	3.33%	12.5%	7.50%	22.5%	2.50%	7.50%
Redis	10.06%	42.50%	3.33%	12.5%	11.88%	37.50%	2.77%	5.00%
Kafka	12.50%	25.00%	7.78%	15.00%	9.44%	35.00%	5.55%	10.00%
Unix	13.33%	40.00%	0.00%	0.00%	0.50%	2.50%	0.00%	0.00%
Spark	1.40%	5.00%	0.42%	2.50%	1.25%	7.50%	0.00%	0.00%

mance on a static copy of the workload or a standard benchmark that simulates a real workload [31]. Common tuning algorithms include heuristic-based rules [32], [33], Bayesian optimization [3], [4], [34], reinforcement learning [6], [7], [23], [35], LLM-assisted optimizer [36]–[38], etc. Despite the effectiveness of these algorithms, offline tuning optimistically assumes that the workload will never shift over time, which rarely holds in practice.

The AutoML community proposes methods for tuning model performance on dynamic datasets, which share a similar spirit with online configuration tuning. DWM [39] maintains an ensemble of models trained on datasets at different time. OAML [40] considers the observations within the sliding window as the same task. AD²ME [41] weights the performance of the same configuration at different time.

VII. CONCLUSION

The demand for tuning configurations of online services that handle dynamic workloads has significantly increased in the industry. This paper presents A-Tune-Online, an online configuration tuning system that delivers superior tuning performance, efficiency, and QoS guarantee simultaneously. A-Tune-Online restarts the optimization based on explicit workload shift detection to eliminate negative historical observations from previous dissimilar workloads. To detect workload shift accurately, we design a multi-stage detection strategy with rule-based and replay-based detectors. To overcome efficiency issue in optimization restart due to limited observations, we utilize warm start. Specifically, we identify similar previous workloads via a regression model and reused the knowledge in history through reusing configurations or surrogate models. To avoid performance degradation and guarantee QoS, we maintain a safety region and suggested the configuration with the best expected improvement from the region. Experimental results on five scenarios showcase that A-Tune-Online accurately detects workload shifts, adapts more quickly to new workloads, and recommends configurations with better overall QoS guarantee, compared to state-of-art tuning systems.

ACKNOWLEDGMENT

This work is supported by National Natural Science Foundation of China (U23B2048, U22B2037), research grant No. SH-2024JK29, and High-performance Computing Platform of Peking University. Bin Cui is the corresponding author.

REFERENCES

- [1] D. Van Aken, A. Pavlo, G. J. Gordon, and B. Zhang, "Automatic database management system tuning through large-scale machine learning," in *Proceedings of the 2017 ACM international conference on management of data*, 2017, pp. 1009–1024.
- [2] Y. Shen, X. Ren, Y. Lu, H. Jiang, H. Xu, D. Peng, Y. Li, W. Zhang, and B. Cui, "Rover: An online spark sql tuning service via generalized transfer learning," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023, pp. 4800–4812.
- [3] R. Wang, Q. Wang, Y. Hu, H. Shi, Y. Shen, Y. Zhan, Y. Fu, Z. Liu, X. Shi, and Y. Jiang, "Industry practice of configuration auto-tuning for cloud applications and services," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 1555–1565.
- [4] X. Zhang, H. Wu, Z. Chang, S. Jin, J. Tan, F. Li, T. Zhang, and B. Cui, "Restune: Resource oriented tuning boosted by meta-learning for cloud databases," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 2102–2114.
- [5] J. Xin, K. Hwang, and Z. Yu, "Locat: Low-overhead online configuration auto-tuning of spark sql applications," in *Proceedings of the 2022 International Conference on Management of Data*, 2022, pp. 674–684.
- [6] G. Li, X. Zhou, S. Li, and B. Gao, "Qtune: A query-aware database tuning system with deep reinforcement learning," *Proceedings of the VLDB Endowment*, vol. 12, no. 12, pp. 2118–2130, 2019.
- [7] B. Cai, Y. Liu, C. Zhang, G. Zhang, K. Zhou, L. Liu, C. Li, B. Cheng, J. Yang, and J. Xing, "Hunter: an online cloud database hybrid tuning system for personalized requirements," in *Proceedings of the 2022 International Conference on Management of Data*, 2022, pp. 646–659.
- [8] Y. Li, H. Jiang, Y. Shen, Y. Fang, X. Yang, D. Huang, X. Zhang, W. Zhang, C. Zhang, P. Chen *et al.*, "Towards general and efficient online tuning for spark," *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 3570–3583, 2023.
- [9] X. Zhang, H. Wu, Y. Li, J. Tan, F. Li, and B. Cui, "Towards dynamic and safe configuration tuning for cloud databases," in *Proceedings of the 2022 International Conference on Management of Data*, 2022, pp. 631–645.
- [10] P.-J. Liu, C.-P. Li, and H. Chen, "Enhancing storage efficiency and performance: A survey of data partitioning techniques," *Journal of Computer Science and Technology*, vol. 39, no. 2, pp. 346–368, 2024.
- [11] P. Chen, B. Xu, H. Li, W. Wang, Y. Peng, S. S. Bhowmick, X. Chen, and J. Cui, "An efficient framework for secure dynamic skyline query processing in the cloud," *Data Science and Engineering*, pp. 1–21, 2024.
- [12] S. Shan, F. Zhao, Y. Wei, and M. Liu, "Disaster management 2.0: A real-time disaster damage assessment model based on mobile social media data—a case study of weibo (chinese twitter)," *Safety science*, vol. 115, pp. 393–413, 2019.
- [13] J. Snoek, H. Larochelle, and R. P. Adams, "Practical bayesian optimization of machine learning algorithms," in *Advances in neural information processing systems*, 2012, pp. 2951–2959.
- [14] F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Sequential model-based optimization for general algorithm configuration," in *International Conference on Learning and Intelligent Optimization*. Springer, 2011, pp. 507–523.
- [15] J. S. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in *Advances in neural information processing systems*, 2011, pp. 2546–2554.
- [16] X. Zhang, Z. Chang, Y. Li, H. Wu, J. Tan, F. Li, and B. Cui, "Facilitating database tuning with hyper-parameter optimization: a comprehensive experimental evaluation," *Proceedings of the VLDB Endowment*, vol. 15, no. 9, pp. 1808–1821, 2022.
- [17] D. R. Jones, M. Schonlau, and W. J. Welch, "Efficient global optimization of expensive black-box functions," *Journal of Global optimization*, vol. 13, no. 4, pp. 455–492, 1998.
- [18] J. Tan, T. Zhang, F. Li, J. Chen, Q. Zheng, P. Zhang, H. Qiao, Y. Shi, W. Cao, and R. Zhang, "ibTune: Individualized buffer tuning for large-scale cloud databases," *Proceedings of the VLDB Endowment*, vol. 12, no. 10, pp. 1221–1234, 2019.
- [19] M. Feurer, A. Klein, K. Eggenberger, J. Springenberg, M. Blum, and F. Hutter, "Efficient and robust automated machine learning," in *Advances in neural information processing systems*, 2015, pp. 2962–2970.
- [20] R. Qin, F. Chen, T. Wang, L. Yuan, X. Wu, Y. Kang, Z. Zhang, C. Zhang, and Y. Yu, "Multi-agent policy transfer via task relationship modeling," *Science China Information Sciences*, vol. 67, no. 8, p. 182101, 2024.
- [21] A. Kopytov, "Sysbench: a system performance benchmark," <http://sysbench.sourceforge.net/>, 2004.
- [22] S. Huang, J. Huang, J. Dai, T. Xie, and B. Huang, "The hiben benchmark suite: Characterization of the mapreduce-based data analysis," in *2010 IEEE 26th International conference on data engineering workshops (ICDEW 2010)*. IEEE, 2010, pp. 41–51.
- [23] J. Zhang, Y. Liu, K. Zhou, G. Li, Z. Xiao, B. Cheng, J. Xing, Y. Wang, T. Cheng, L. Liu *et al.*, "An end-to-end automatic cloud database tuning system using deep reinforcement learning," in *Proceedings of the 2019 International Conference on Management of Data*, 2019, pp. 415–432.
- [24] Y. Li, Y. Shen, W. Zhang, Y. Chen, H. Jiang, M. Liu, J. Jiang, J. Gao, W. Wu, Z. Yang, C. Zhang, and B. Cui, "Openbox: A generalized black-box optimization service," *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 2021.
- [25] openEuler, "<https://www.openeuler.org/>."
- [26] J. Li, Q. Chen, X.-X. Tang, and M.-Y. Guo, "Sha: Qos-aware software and hardware auto-tuning for database systems," *Journal of Computer Science and Technology*, vol. 39, no. 2, pp. 369–383, 2024.
- [27] S. Huang, Y. Qin, X. Zhang, Y. Tu, Z. Li, and B. Cui, "Survey on performance optimization for database systems," *Science China Information Sciences*, vol. 66, no. 2, p. 121102, 2023.
- [28] A. Fekry, L. Carata, T. Pasquier, A. Rice, and A. Hopper, "Tuneful: An online significance-aware configuration tuner for big data analytics," *arXiv preprint arXiv:2001.08002*, 2020.
- [29] Y. Huang, C. Li, M. Chen, and Z. Su, "Act-sagan: Automatic configuration tuning for kafka with self-attention generative adversarial networks," in *Proceedings of the 5th International Conference on Computer Science and Software Engineering*, 2022, pp. 180–184.
- [30] A. Renda, Y. Chen, C. Mendis, and M. Carbin, "DiffTune: Optimizing cpu simulator parameters with learned differentiable surrogates," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 442–455.
- [31] X. Zhang, H. Wu, Y. Li, Z. Tang, J. Tan, F. Li, and B. Cui, "An efficient transfer learning based configuration adviser for database tuning," *Proceedings of the VLDB Endowment*, vol. 17, no. 3, pp. 539–552, 2023.
- [32] S. Venkataraman, Z. Yang, M. Franklin, B. Recht, and I. Stoica, "Ernest: Efficient performance prediction for large-scale advanced analytics," in *13th {USENIX} symposium on networked systems design and implementation ({NSDI} 16)*, 2016, pp. 363–378.
- [33] S. Sidhanta, W. Golab, and S. Mukhopadhyay, "Deadline-aware cost optimization for spark," *IEEE Transactions on Big Data*, vol. 7, no. 1, pp. 115–127, 2019.
- [34] K. Kanellis, C. Ding, B. Kroth, A. Müller, C. Curino, and S. Venkataraman, "Llamatune: sample-efficient dbms configuration tuning," *Proceedings of the VLDB Endowment*, vol. 15, no. 11, pp. 2953–2965, 2022.
- [35] D. Van Aken, D. Yang, S. Brillard, A. Fiorino, B. Zhang, C. Bilien, and A. Pavlo, "An inquiry into machine learning-based automatic configuration tuning services on real-world database management systems," *Proceedings of the VLDB Endowment*, vol. 14, no. 7, pp. 1241–1253, 2021.
- [36] J. Lao, Y. Wang, Y. Li, J. Wang, Y. Zhang, Z. Cheng, W. Chen, M. Tang, and J. Wang, "Gptuner: A manual-reading database tuning system via gpt-guided bayesian optimization," *Proceedings of the VLDB Endowment*, vol. 17, no. 8, pp. 1939–1952, 2024.
- [37] I. Trummer, "Db-bert: a database tuning tool that reads the manual," in *Proceedings of the 2022 international conference on management of data*, 2022, pp. 190–203.
- [38] X. Zhou, Z. Sun, and G. Li, "Db-gpt: Large language model meets database," *Data Science and Engineering*, vol. 9, no. 1, pp. 102–111, 2024.
- [39] R. Bakirov, D. Fay, and B. Gabrys, "On-the-fly learning of adaptive strategies with bandit algorithms," in *8th ICML Workshop on Automated Machine Learning (AutoML)*, 2021.
- [40] B. Celik, P. Singh, and J. Vanschoren, "Online automl: An adaptive automl framework for online learning," *Machine Learning*, vol. 112, no. 6, pp. 1897–1921, 2023.
- [41] S. Lu, Y.-H. Zhou, J.-C. Shi, W. Zhu, Q. Yu, Q.-G. Chen, Q. Da, and L. Zhang, "Non-stationary continuum-armed bandits for online hyperparameter optimization," in *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*, 2022, pp. 618–627.